

APIs de LLM e o primeiro agente em loop

HTTP, SDKs, janela de contexto, tool calling e o laço que transforma modelo em agente

fetch → SDK janela de contexto tool calling agent loop HITL

Prof. Me. Élder F. F. Bernardi · IF Sul Câmpus Passo Fundo

Posição no percurso da disciplina

MAPA DA DISCIPLINA

- **Aula 02** — a teoria do laço: tool calling, laço de controle, critérios de parada.
- **Hoje** — construir esse laço **na mão**, em código, contra uma API real.

A regra do percurso: primeiro entender o que o harness automatiza; só depois usá-lo.

Objetivos da aula

1. **Chamar APIs de LLM** — os dois padrões de indústria (OpenAI e Anthropic) e o wire format por baixo dos SDKs.
2. **Gerir entrada e saída** — histórico de mensagens e janela de contexto, vistos e medidos em código.
3. **Construir um agente em loop** — objetivo, iteração, dispatch de tools e parada validável.
4. **Colocar o humano no loop** — gate de aprovação (HITL) como parte do IO do agente.

Material (código da aula): github.com/elderbernardiifsul/prog-agentica-helloLLM

Verificação do ambiente

FEITO EM CASA (GUIA DE PREPARAÇÃO)

```
git clone https://github.com/elderbernardiifsul/prog-agentica-helloLLM.git
cd prog-agentica-helloLLM
cp .env.example .env          # cole sua GEMINI_API_KEY
npm install
npm run 01                    # imprimiu JSON? pronto.
```

- Chave gratuita: <https://aistudio.google.com/apikey>
- Node ≥ 20.6 (`node --version`)
- Alternativa 100% local: Ollama (`qwen2.5:7b`) — mesmo código, outra `baseUrl`.

Quem travou aqui: resolver AGORA, em dupla — sem setup não há mini-lab.

Revisão: o laço de controle

AULA 02, SLIDE 23 – HOJE CADA SETA VIRA CÓDIGO

decisão → **ação** → **observação** → **estado**

- O **modelo** decide (propõe texto ou pedido de ferramenta).
- O **runtime** age (executa a ferramenta de verdade).
- O resultado volta como **observação** (nova mensagem no contexto).
- O **estado** determina: continuar, ou parar por sucesso / limite / erro.

Hoje: runtime = ~80 linhas de JavaScript escritas por vocês.

Estrutura da requisição

DEGRAU 01 • 0 REQUEST

```
POST /chat/completions
Authorization: Bearer <chave>
Content-Type: application/json
```

```
{
  "model": "gemini-2.5-flash",
  "messages": [
    { "role": "system", "content": "Você é um assistente conciso." },
    { "role": "user", "content": "O que é uma janela de contexto?" }
  ]
}
```

Três roles até aqui: **system** (regras), **user** (pedido), **assistant** (respostas do modelo).

Estrutura da resposta

DEGRAU 01 • 0 RESPONSE

```
{
  "choices": [{
    "message": { "role": "assistant", "content": "É o limite de tokens..." },
    "finish_reason": "stop"
  }],
  "usage": { "prompt_tokens": 28, "completion_tokens": 31, "total_tokens": 59 }
}
```

- `choices[0].message` — a resposta (entra no histórico).
- `finish_reason` — por que parou: `stop`, `tool_calls`, `length`.
- `usage` — **o custo**. Cada token de entrada e saída é cobrado.

Demonstração: chamada HTTP direta (degrau 01)

REPO PROG-AGENTICA-HELLOLLM · 01-FETCH-CRU.MJS

```
npm run 01
```

O que observar:

- É um `fetch` comum — sem SDK.
- O JSON inteiro da resposta: localizar `content`, `finish_reason`, `usage`.
- Rodar duas vezes: a resposta muda (amostragem), o formato permanece.

Padrões de API: OpenAI e Anthropic

OPENAI CHAT COMPLETIONS × ANTHROPIC MESSAGES

Aspecto	OpenAI	Anthropic
Endpoint	POST /chat/completions	POST /v1/messages
Auth	Authorization: Bearer	x-api-key
System prompt	mensagem na lista	parâmetro system próprio
max_tokens	opcional	obrigatório
Resposta	content string	content = lista de blocos
Tool: declaração	function.parameters	input_schema
Tool: pedido	tool_calls + role tool	bloco tool_use + bloco tool_result
Usage	prompt/completion_tokens	input/output_tokens

Mesmos conceitos, formatos diferentes. Código comentado: 02b-sdk-anthropic.mjs.

Compatibilidade entre provedores

POR QUE A AULA USA GEMINI COM SDK OPENAI

Gemini, Ollama, Mistral e vLLM expõem endpoints **compatíveis com o formato OpenAI**.

```
const client = new OpenAI({  
  apiKey: process.env.GEMINI_API_KEY,  
  baseURL: "https://generativelanguage.googleapis.com/v1beta/openai",  
});
```

Trocar de provedor = trocar `baseURL` (e a chave). O código não muda.

É a mesma razão pela qual aprender **um** wire format rende: ele é o dialeto comum do ecossistema.

Demonstração: o SDK e suas abstrações (degrau 02)

REPO PROG-AGENTICA-HELLOLLM · 02-SDK-OPENAI.MJS

```
const resposta = await client.chat.completions.create({ model, messages });
```

O SDK fez por você: URL + headers, JSON, **retries com backoff** (429/5xx), timeouts, erros tipados.

O que ele **não** fez: decidir o que entra em `messages`, quando parar, o que fazer com a resposta. Isso é o seu programa — e é o assunto do resto da aula.

O outro formato: `npm run 02b` — o contraste Anthropic, comentado linha a linha.

Frameworks de agentes: escopo e adiamento

LANGCHAIN · LANGGRAPH · VERCEL AI SDK

- São **embalagens do loop** que vamos construir hoje: abstraem histórico, tools, dispatch e parada.
- Úteis em produção; péssimos como primeira exposição — escondem exatamente o que precisamos entender.
- Depois de hoje, vocês saberão **o que** eles abstraem — e poderão julgar **quando** valem a pena.

Hoje: zero frameworks. SDK + código nosso.

Gestão do histórico de mensagens

DEGRAU 03 · O MODELO NÃO LEMBRA DE NADA

```
const historico = [{ role: "system", content: "..."}];  
// a cada turno:  
historico.push({ role: "user", content: entrada });  
const resposta = await client.chat.completions.create({ model, messages: historico });  
historico.push(resposta.choices[0].message); // ← sem isto, "amnésia"
```

A "memória" do chat **é este array**. A API não guarda nada entre chamadas.

Esquecer o push do assistant = modelo que não lembra do que ele mesmo disse.

Demonstração: chat com histórico (degrau 03)

REPO · PROG-AGENTICA-HELLOLLM · 03-CHAT-READLINE.MJS

```
npm run 03
```

Roteiro:

1. meu nome é <nome> — depois pergunte algo qualquer, depois qual é meu nome? → ele lembra (o array funciona).
2. Observar a linha de status: prompt_tokens **cresce a cada turno.**

Pergunta pra turma: quem está pagando pelos tokens do histórico reenviado?

Montagem do prompt e janela de contexto

DEGRAU 03B • MONTAGEM DO PROMPT

A cada chamada, o modelo vê **um único texto montado**:

```
[ system ] + [ histórico inteiro ] + [ tool defs ] + [ mensagem nova ]  
           ↓  
        modelo  
           ↓  
       resposta
```

- A API é **stateless**: tudo é reenviado, sempre.
- A **janela de contexto** é o limite de tokens dessa montagem (+ resposta).

Limites da janela e estratégias de gestão

- Modelos atuais: de ~128K a ~1M tokens de janela.
- Quando estoura: erro da API, ou o provedor corta — **algo fica de fora**.
- Quem decide o que fica é o **seu código**: trimming (cortar o mais antigo), sumarização, compaction.
- Gancho da aula 02: é aqui que a **engenharia de contexto** deixa de ser abstrata.

Custo mora no mesmo lugar: histórico longo = `prompt_tokens` alto **em toda chamada**.

Demonstração: limite da janela de contexto (degrau 03b)

REPO PROG-AGENTICA-HELLOLLM · 03B-JANELA-CONTEXTO.MJS · JANELA SIMULADA DE 250 TOKENS

```
npm run 03b
```

1. meu nome é <nome>
 2. Peça 2-3 respostas longas ("explique HTTP em detalhes").
 3. Linhas ✂️ caiu para fora da janela começam a aparecer.
 4. qual é meu nome? → **ele esqueceu**. O nome saiu da janela.
- O dump ┌ CONTEXTO QUE SERÁ ENVIADO ─ mostra exatamente o que o modelo vê.

Tool calling: fronteira entre modelo e runtime

DEGRAU 04 · A FRONTEIRA MODELO / RUNTIME (AULA 02, SLIDE 19)

- O modelo **emite um pedido estruturado**: "chamem `somar` com `{a: 2, b: 3}`".
- O **seu código** decide executar, executa, e devolve o resultado como mensagem.
- O modelo **nunca toca o mundo** — nem arquivos, nem rede, nem terminal.

Consequência: todo poder do agente é poder que **você** deu, tool por tool. (Menor privilégio, aula 02.)

Declaração de ferramentas por schema

UM SCHEMA JSON – É TUDO QUE O MODELO VÊ

```
{
  type: "function",
  function: {
    name: "somar",
    description: "Soma dois números com precisão exata.",
    parameters: {
      type: "object",
      properties: { a: { type: "number" }, b: { type: "number" } },
      required: ["a", "b"],
    },
  },
}
```

A `description` é interface **para o modelo**: escreva-a para ele saber *quando* usar a tool.

O ciclo de tool calling

UMA VOLTA, QUATRO MENSAGENS

1. `user: "Quanto é 123456 + 654321? Use a ferramenta."`
2. `assistant: tool_calls: [{ id, function: { name: "somar", arguments: '{"a":123456,...}' } } ] — finish_reason: "tool_calls"`
3. **Seu código:** `JSON.parse(arguments)` → executa → `{ role: "tool", tool_call_id: id, content: "777777" }`
4. `assistant: "O resultado é 777.777."`

Tudo isso **entra no histórico** — o ciclo é feito de mensagens.

Demonstração: ciclo manual de tool calling (degrau 04)

REPO PROG-AGENTICA-HELLOLLM · 04-TOOL-CALLING.MJS

```
npm run 04
```

O que observar:

- `finish_reason: "tool_calls"` — o modelo **parou para pedir**.
- `arguments` chega como **string** → `JSON.parse` + validação sempre (o modelo pode mandar lixo).
- O `tool_call_id` amarra pedido e resultado.
- Segunda chamada: o modelo lê o resultado e responde ao usuário.

Requisitos para construir um agente

TRÊS INGREDIENTES

1. **Objetivo** — uma missão no `system / user` ("adivinhe o número"; "monte o projeto").
 2. **Iteração** — um `while` em volta do ciclo do degrau 04: agir, observar, agir de novo.
 3. **Parada válida** — condição objetiva para `sucesso`, `limite` e `erro`.
- Agente = modelo + tools + **loop com critérios de parada**. É isso. O resto é sofisticação.

Estrutura do agent loop

O CORAÇÃO, EM 12 LINHAS

```
let estadoFinal = null; // "sucesso" | "limite" | "erro"
let passo = 0;
while (estadoFinal === null) {
  passo++;
  if (passo > MAX_PASSOS) { estadoFinal = "limite"; break; }
  const r = await client.chat.completions.create({ model, messages, tools });
  const msg = r.choices[0].message;
  messages.push(msg);
  if (!msg.tool_calls) { /* modelo conversou: reorientar e continuar */ continue; }
  for (const call of msg.tool_calls) {
    /* dispatch: executar, tracear, push do tool result, checar sucesso */
  }
}
```

Critérios de parada

SEM ELES, NÃO HÁ AGENTE – HÁ UM GERADOR DE CUSTO

Estado	Gatilho	Sem ele...
sucesso	objetivo verificado (tool finalizar , checagem)	agente não sabe que acabou
limite	passo > MAX_PASSOS	loop infinito, conta infinita
erro	exceção da API / tool irrecuperável	crash sem trace

E o caso chato: o modelo **conversa em vez de agir** → reinjetar instrução ("continue: use a tool") e seguir o loop.

Registro de execução (trace)

FORMATO DA DISCIPLINA

```
[passo 3] chute 50 -> maior  
[passo 4] chute 75 -> menor  
[passo 5] chute 62 -> acertou
```

```
== FIM: sucesso em 5 passo(s) ==
```

- Uma linha por ação: [passo N] ação -> observação .
- Sem trace não há depuração, não há auditoria, não há evidência.
- É o que você entrega na atividade EaD — e o que os harnesses geram sozinhos (aula 04).

O loop do jogo

05-loop-jogo.mjs (repo prog-agentica-helloLLM) — o agente deve adivinhar o número secreto (1..100) em até 10 chutes. O loop está pronto pela metade: completem os TODOs.

TODO 1: parada por limite (sem ela o loop roda pra sempre — Ctrl+C). TODO 2: dispatch (parse, executar, trace, tool result). TODO 3: parada por sucesso. Rodou? Desafios: MAX_PASSOS = 3; proibir busca binária no system prompt e comparar traces.

Síntese do mini-laboratório

MAPA DO LAB PARA O DIAGRAMA

- `while (estadoFinal === null)` — o **laço de controle** inteiro.
- Chamada à API — a **decisão** do modelo.
- `chutar()` no seu código — a **ação** no mundo.
- Push do tool result — a **observação** que realimenta o contexto.
- `estadoFinal` — o **estado** que decide parar.

Pergunta: onde entraria uma **segunda tool**? (Resposta prática: daqui a pouco, no agente de arquivos.)

Intervenção humana no laço (HITL)

HUMAN-IN-THE-LOOP · POR QUÊ E ONDE

- **Por quê:** ações irreversíveis (escrever, apagar, pagar), menor privilégio, e a regra da disciplina — *o estudante responde pelo resultado entregue*.
- **Onde:** um gate **antes do efeito** — o pedido da tool é mostrado ao humano; só executa com aprovação.
- A recusa **não encerra o agente**: vira observação (`NEGADO` pelo humano: `<motivo>`) que o modelo usa para replanejar.

Aprovar · corrigir · interromper — os três verbos da intervenção (matriz de competências).

Demonstração: gate de aprovação humana (degrau 07)

REPO PROG-AGENTICA-HELLOLLM · 07-HITL.MJS · MISSÃO: ESCREVER UM HAICAI

```
npm run 07
```

Rodada 1 — aprovar (s): arquivo escrito, missão concluída.

Rodada 2 — negar (n) com motivo *"quero em inglês"*:

```
[passo 2] [INTERVENÇÃO] escrita negada: quero em inglês  
[passo 3] escrever({"caminho":"poema.md",...}) -> ok: poema.md
```

O agente **leu o motivo e ajustou**. O humano é parte do IO.

Atividade EaD — agente de arquivos

PRAZO: 2 AULAS • ENUNCIADO COMPLETO: [DOCS/ATIVIDADE-EAD-AGENTE-ARQUIVOS.MD](#) NO REPO

Missão: montar um pacote Node (`workspace-demo`) numa sandbox, com dependência instalada pelo próprio agente (`npm install` via tool `executar`) — verificado por script (`npm run verificar`). Desejável: `hello.mjs` imprimindo "Hello Agent!", executado pelo agente com evidência no trace.

1. **Núcleo** — completar TODOs 1..4 de `06-agente-arquivos/agente.mjs` até **MISSÃO CUMPRIDA** .
 2. **Extensões obrigatórias** — gate HITL em `escrever` e `executar` (com 1 intervenção real no trace) + **uma tool nova de autoria própria** (uma tool é só uma função + o schema que a descreve).
 3. **Registro** — `relatorio.md`: trace comentado, decisão da tool nova, autonomia.
- Trace inventado ou verificador adulterado = atividade zerada.

Autonomia

FECHAMENTO

- Do loop que você escreveu hoje: o que sabe reescrever **do zero, sem assistente?**
- O que muda no seu código se trocarmos `gemini-2.5-flash` por outro modelo? E o que **não** muda?
- Onde o custo mora? (Duas respostas certas: histórico reenviado; loop sem limite.)

Aula 04: OpenCode — loop, tools, permissões, trace e HITL de hoje, embalados num harness profissional. Vocês vão reconhecer cada peça.

Nota sobre produção com IA generativa

TRANSPARÊNCIA

- A **direção pedagógica**, as **ideias de conteúdo**, a **sequência didática** e as **atividades** desta aula são do professor.
- O material (texto, código e slides) foi **produzido com apoio de IA generativa**, em sessão com forte **HITL**: supervisão contínua, decisões, revisões e ajustes diretos no texto pelo professor.
- Coerência com a disciplina: é exatamente o fluxo de trabalho que estudamos — agente produz, humano dirige, intervém e responde pelo resultado.

Referências

- OpenAI — Chat Completions e Function calling: <https://platform.openai.com/docs>
- Anthropic — Messages API e Tool use: <https://docs.anthropic.com>
- Google — Gemini API, compatibilidade OpenAI: <https://ai.google.dev/gemini-api/docs/openai>
- Ollama — OpenAI compatibility: <https://docs.ollama.com>
- Yao et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*.
- Código, material de apoio e enunciado da atividade: <https://github.com/elderbernardiifsul/prog-agentica-helloLLM>
- Guia de estudo da aula 02 (repositório da disciplina).